

Lion Guardians Vehicles Report

Technical Guide

Vehicle Movement Analysis — Methodology & Calculation Reference
Version 1.0 · Generated April 15, 2026

1. Overview

The **LG Vehicles Report** workflow analyses GPS-tracked vehicle movement in the Amboseli ecosystem for the **Lion Guardians** programme. It ingests telemetry from **EarthRanger** for the *Vehicles* subject group, computes speed maps, vehicle tracks, and a speed-over-time line chart per subject, generates speed and distance statistics, and delivers an interactive dashboard plus a print-ready Word report.

The workflow produces two map visualisations (Speed Map and Vehicle Tracks), a speed line chart, four scalar metric widgets, and a per-subject summary CSV for each tracked vehicle.

Note: The grouper is fixed to `subject_name` — one set of outputs is produced per individual vehicle.

2. Dependencies & Prerequisites

2.1 EarthRanger Connection

All telemetry is fetched from an **EarthRanger** instance via `set_er_connection`. The workflow calls `get_subjectgroup_observations` for the subject group **Vehicles** with filter: `clean`, discarding junk-flagged observations before they reach the pipeline. Subject details and subject-source details are not included in the raw fetch (`include_details: false`).

2.2 Grouper

The grouper is pre-configured to `subject_name` and is not user-selectable. All downstream map, metric, and report tasks iterate over individual vehicles.

2.3 Static Geodata Files

Three boundary datasets are downloaded from Dropbox and cached locally:

Dataset	File	Purpose
Group Ranch Boundaries	<code>lg_group_ranch_boundaries.gpkg</code>	Community ranch polygons in Amboseli
Conflict Hotspot Areas	<code>lg_conflict_hotspots.gpkg</code>	Known human–lion conflict hotspot features
Protected Areas	<code>lg_protected_areas.gpkg</code>	National parks and reserves

All three files use `overwrite_existing: false` (3 retries). After loading, each is reprojected to **EPSG:4326** and annotated with its geometry type before layer creation.

2.4 Word Document Templates

Template file	Purpose
<code>vehicles_cover_page.docx</code>	Report cover page — subject count, time range, preparer
<code>custom_vehicle_template.docx</code>	Per-subject section — speed map, tracks map, line chart, summary table

Both templates are downloaded from Dropbox with 2 retries.

2.5 Base Map Tile Layers

Layer	Opacity	Max zoom
ArcGIS World Hillshade	100 %	20
ArcGIS World Street Map	15 %	20

The hillshade provides full-opacity terrain context. The street map is overlaid at 15 % to show roads and settlement names without obscuring the vehicle data layers.

3. Data Ingestion Pipeline

3.1 Observations → Relocations

`process_relocations` converts raw EarthRanger observations to a standardised GeoDataFrame. Retained columns:

Column	Source field	Description
<code>groupby_col</code>	<code>internal</code>	Subject identifier for grouping
<code>fixtime</code>	<code>observation timestamp</code>	UTC datetime of the GPS fix
<code>junk_status</code>	<code>EarthRanger flag</code>	True if fix is marked junk
<code>geometry</code>	<code>lat/lon</code>	Point geometry (WGS 84)
<code>extra__subject__name</code>	<code>subject.name</code>	Vehicle display name
<code>extra__subject__subject_subtype</code>	<code>subject.subject_subtype</code>	Subject subtype
<code>extra__subject__sex</code>	<code>subject.sex</code>	Subject sex field

Three bad coordinate pairs are removed unconditionally:

- (180.0, 90.0) — boundary sentinel
- (0.0, 0.0) — null-island artefact
- (1.0, 1.0) — common default / test value

Cleaned relocations are persisted as `vehicle_relocations.geoparquet`.

3.2 Relocations → Trajectories

`relocations_to_trajectory` connects consecutive fixes per vehicle into LineString segments, adding `dist_meters`, `speed_kmh`, `segment_start`, and `segment_end`. Trajectories are persisted as `vehicle_trajectories.geoparquet`.

3.3 Temporal Index, Speed Classification & Column Renaming

`add_temporal_index` keys the trajectory GeoDataFrame to `segment_start`, grouped by `subject_name`. `apply_classification` then bins `speed_kmh` into **6 equal-interval classes** (output column: `speed_bins`, labels to 1 decimal place). Three columns are renamed via `map_columns`:

Original column	Renamed to
<code>extra__name</code>	<code>subject_name</code>
<code>extra__sex</code>	<code>subject_sex</code>

Original column	Renamed to
extra__subject_subtype	subject_subtype

The renamed GeoDataFrame is split into per-vehicle partitions by `split_groups`. All downstream map and metric tasks iterate over these partitions via `mapvalues`.

4. Static Map Layers

Four static layers are built once and composited onto every vehicle-level map to provide spatial context.

4.1 Layer Styles

Layer	Colour (RGB)	Opacity	Filled	Notes
Group Ranch Boundaries	(169, 169, 169) grey	55 %	No	Outline only, line width 4.5
Conflict Hotspots	(220, 20, 60) crimson	75 %	Yes	Point radius 2.55, line width 1.95
Protected Areas	(77, 102, 0) dark green	35 %	Yes	Line width 1.95
Hotspot Text Labels	(20, 20, 20) near-black	—	—	Arial, 1 000 m base, 40–75 px clamp, centroid-anchored

5. Map Outputs — Methodology

5.1 Speed Map

The speed map renders each trajectory segment coloured by its speed bin. Processing steps before layer creation:

- Sort segments ascending by `speed_bins` so faster segments render on top.
- Apply a custom 6-colour diverging palette to `speed_bins`:

Bin (slowest → fastest)	Hex colour	Visual tone
1 — slowest	#1a9850	Dark green
2	#91cf60	Light green
3	#d9ef8b	Yellow-green
4	#fee08b	Yellow
5	#fc8d59	Orange
6 — fastest	#d73027	Red

- Format bin labels and speed values to 1 decimal place with km/h unit.
- Retain only: `dist_meters`, `speed_bins_colormap`, `geometry`, `speed_kmh`, `speed_bins_formatted`.

`create_path_layer` renders the filtered GeoDataFrame with width 1.55 px (min 2 px, max 8 px, screen-space), rounded caps and joins, 55 % opacity. Legend title: *Speed (km/h)*. The layer is combined with the four static boundary layers. The map is auto-zoomed to the trajectory extent and persisted as HTML (suffix: `speedmap`), then converted to PNG at 2× scale with a 40 s tile-load wait.

5.2 Vehicle Tracks Map

create_path_layer renders all trajectory segments per vehicle as a uniform blue path:

Property	Value
Colour	RGB(0, 0, 255) — blue
Width	1.55 px, min 2 px, max 8 px (screen-space)
Cap / Join	Rounded
Opacity	55 %
Max zoom	15 (draw_map setting)

The path layer is combined with the four static layers. The view state is shared with the Speed Map (same envelope and zoom). The map is persisted as HTML (suffix: tracks) and converted to PNG at 2× scale with a 40 s tile-load wait.

5.3 Speed Line Chart

draw_line_chart plots vehicle speed over the analysis period for each subject individually (no category grouping):

Property	Value
X axis	segment_start — formatted as YYYY-MM-DD, ticks rotated −45°
Y axis	speed_kmhr — labelled Speed (km/h), 1 d.p. ticks
Line shape	Linear (no smoothing)
Background	#f5f5f5 plot area, #e0e0e0 gridlines
Hover mode	x unified
Legend	Hidden

The chart is persisted as HTML (suffix: speed_line_chart) and converted to PNG via html_to_png (2× scale, 10 ms wait). Widget title: *Vehicle Speed Over Time*.

6. Summary Metrics

6.1 Per-Vehicle Summary Table

summarize_df aggregates trajectory data grouped by subject_name:

Output column	Source	Aggregator	Unit	D.p.
mean_speed	speed_kmhr	mean	km/h	2
min_speed	speed_kmhr	min	km/h	2
max_speed	speed_kmhr	max	km/h	2
total_distance	dist_meters	sum	m → km	2

add_totals_row appends a *Total* row across all vehicles. The table is persisted as CSV.

6.2 Scalar Dashboard Widgets

Four scalar values are derived per group by summing the per-subject column from the summary table via `dataframe_column_sum`, then rounded to 2 d.p. by `round_off_values`:

Widget title	Source column	Unit
Mean Speed	mean_speed	km/h
Min Speed	min_speed	km/h
Max Speed	max_speed	km/h
Distance covered	total_distance	km

7. Word Report (.docx)

7.1 Cover Page

`create_cl_ctx_cover` builds the cover context using the count of unique subjects (`dataframe_column_nunique` on `groupby_col`), the time range, and *Ecoscope* as preparer. `create_context_page_lg` renders it into `lg_cover_page.docx`.

7.2 Per-Vehicle Sections

`create_vehicles_grouper_ctx` assembles a context dict per vehicle containing the trajectory DataFrame, summary table CSV, Speed Map PNG, Tracks Map PNG, and Speed Line Chart PNG. `create_grouper_page` renders each section from the `custom_vehicle_template.docx` template. Image boxes: **8.28 × 14.15 cm**. `validate_images`: `true` catches missing PNGs before rendering.

7.3 Document Merge

`merge_cl_files` concatenates the cover page and all per-vehicle sections into a single Word file saved to the results directory.

8. Interactive Dashboard

`gather_dashboard` assembles the vehicles dashboard from seven widget groups:

Widget	Type	Source task
Mean Speed	Single value	<code>round_mean_speed</code> → <code>total_mean_speed_sv_widget</code>
Min Speed	Single value	<code>round_min_speed</code> → <code>total_min_speed_sv_widget</code>
Max Speed	Single value	<code>round_max_speed</code> → <code>total_max_speed_sv_widget</code>
Distance covered	Single value	<code>round_total_distance</code> → <code>total_distance_sv_widget</code>
Speed Map	Map	<code>draw_speedmap</code> → <code>merge_speedmap_widgets</code>
Vehicle Tracks	Map	<code>draw_track_map</code> → <code>merge_tracks_widgets</code>
Vehicle Speed Over Time	Line chart	<code>draw_speed_line_chart</code> → <code>grouped_plot_widget_merge</code>

Note: Single-value and chart widget tasks use `skipif`: `[never]` so the dashboard always assembles, even when some vehicles have no data.

9. Output Files

All files are written to `$ECOSCOPE_WORKFLOWS_RESULTS`.

File / pattern	Format	Content
vehicle_relocations.geoparquet	GeoParquet	Cleaned GPS fix locations
vehicle_trajectories.geoparquet	GeoParquet	Segments with speed_kmhr, dist_meters, speed_bins
_speedmap.html	HTML	Interactive speed map per vehicle
_tracks.html	HTML	Interactive tracks map per vehicle
_speed_line_chart.html	HTML	Interactive speed-over-time line chart
_speedmap.png	PNG	2× screenshot of speed map
_tracks.png	PNG	2× screenshot of tracks map
_speed_line_chart.png	PNG	2× screenshot of speed line chart
_summary.csv	CSV	Speed and distance summary table with totals row
lg_cover_page.docx	Word	Rendered report cover page
.docx	Word	Per-vehicle report section
.docx	Word	Final combined Word report

10. Workflow Execution Logic

10.1 Skip Conditions

Two default skip conditions apply to every task (task-instance-defaults):

- **any_is_empty_df** — skips the task (and all dependants) when any input DataFrame is empty, handling vehicles with no fixes gracefully.
- **any_dependency_skipped** — propagates skips downstream automatically.

Single-value widget and chart widget tasks override this with skipif: [never] to ensure the dashboard always assembles.

10.2 Data Flow Summary

Stage	Tasks
Setup	ER connection, time range, grouper (subject_name), base maps
Geodata download	3 boundary files + 2 Word templates from Dropbox
Static layers	Ranch, hotspot, protected area, hotspot text layers
Telemetry ingest	Observations → relocations → trajectories → temporal index → speed bins → rename → split
Speed Map branch	Sort → 6-colour palette → format labels → filter cols → path layer → compose → zoom → HTML → PNG → widget

Stage	Tasks
Tracks branch	Blue path layer → compose → draw map (zoom 15) → HTML → PNG → widget
Line chart branch	draw_line_chart → HTML → PNG → plot widget
Metrics branch	summarize_df → totals row → CSV; 4 scalar widgets (mean/min/max speed, distance)
Report assembly	Unique subject count → cover page + per-vehicle sections → merge docx
Dashboard	gather_dashboard combines all 7 widgets

11. Software Versions

Package	Version	Role
ecoscope-workflows-core	0.22.17.*	Core task library and workflow engine
ecoscope-workflows-ext-ecoscope	0.22.17.*	Spatial analysis tasks (relocations, trajectories, classification)
ecoscope-workflows-ext-custom	0.0.40.*	Utility tasks (column mapping, screenshots, line chart)
ecoscope-workflows-ext-ste	0.0.18.*	Summary table and totals-row tasks
ecoscope-workflows-ext-mnc	0.0.7.*	MNC domain tasks
ecoscope-workflows-ext-icf	0.0.0.*	ICF domain tasks
ecoscope-workflows-ext-big-life	0.0.8.*	Big Life Foundation domain tasks
ecoscope-workflows-ext-lion-guardians	0.0.6.*	Lion Guardians Word report rendering tasks

Packages are distributed via the prefix.dev conda channel and pinned to patch-compatible versions (*. suffix). The runtime environment is managed by **pixi**.